CSA0920 - PROGRAMMING WITH JAVA

ASSIGNMENT REPORT

https://github.com/192521143simats-sketch/CSA09-192521143-Assignment1

Submitted by

R SARVESH (192521143)

Under the Supervision of

VIJAYARAGAVAN



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

OCTOBER 2026

Task 1 — Understand the Problem and Identify Objects

1.1 Inputs and outputs

Inputs:

- Vehicle registration number.
- Vehicle type: two-wheeler, car or electric vehicle.
- Floor and slot information.
- Entry time and exit time.
- User type: student, staff or visitor.
- Monthly pass details.
- Charging units used by electric vehicles.

Outputs:

- Allocated parking floor and slot.
- Unique parking ticket number.
- Vehicle entry and exit details.
- Parking duration.
- Parking fee.
- Slot availability and error messages.

1.2 Candidate classes

Class	Responsibility
Vehicle	Stores common vehicle details.
TwoWheeler	Represents a two-wheeler and its hourly rate.
Car	Represents a car and its hourly rate.
ElectricCar	Represents an electric car with charging facilities.
ParkingSlot	Stores slot number, floor and availability.
ParkingTicket	Stores ticket number, vehicle and entry time.
ParkingFacility	Allocates and releases parking slots.
ParkingUser	Stores user details and pass information.
FeeCalculator	Calculates parking fees.

1.3 Nouns and verbs

- Nouns become classes or objects: vehicle, ticket, slot, floor, user and facility.
- Verbs become methods: allocate, release, calculate fee, enter, exit and validate.

1.4 Business rules

1. One parking slot can contain only one vehicle at a time.
2. A vehicle cannot be allocated to an occupied slot.
3. Every parking ticket must have a unique ticket number.
4. A released slot must become available again.
5. Invalid floor and slot numbers must be rejected.
6. Parking duration cannot be negative.
7. Students and staff with valid monthly passes park free.
8. Visitors must pay the applicable parking fee.

9. Electric vehicle charging fees must be calculated separately.
10. Duplicate active registrations must be rejected.

1.5 Assumptions

- Every vehicle has a unique registration number.
- Parking rates are fixed for the duration of a parking session.
- Each floor can have a different number of slots.
- Parking time is measured in minutes.
- Every started hour is charged as a complete hour.
- A monthly pass must be valid when the vehicle enters.

1.6 Alternative concept and complexity analysis

If plain parallel arrays were used, the data would be stored in arrays such as regNo[], type[] and entryTime[].

Comparison	Vehicle objects	Parallel arrays
Search by registration	O(n)	O(n)
Add a new attribute	Add a field to the class	Add another array and keep indexes aligned
Space	O(n) for n vehicles	O(n) for n vehicles
Maintenance	Easier to organise	More chances of mismatched data

Task 2 — Choose the Data Representation

2.1 Java data types

Information	Data type	Reason
Floor number	int	Whole number
Slot number	int	Whole number
Registration number	String	Contains letters and numbers
Ticket number	long	Supports a large sequence of tickets
Entry time	LocalDateTime	Stores date and time
Duration	long	Stores minutes
Parking fee	double	Supports decimal amounts
Slot availability	boolean	True or false

2.2 Array representation

Use a jagged two-dimensional array:

String[][] slots;

Each row represents a floor. Each row can have a different number of slots.

For example:

```
String[][] slots = new String[3][];
slots[0] = new String[20];
slots[1] = new String[30];
slots[2] = new String[50];
```

This represents three floors containing 20, 30 and 50 slots respectively. A null value represents a free slot.

2.3 Type conversion and precision

Suppose a car costs ₹30 per hour and is parked for 90 minutes.

Incorrect calculation:

```
int hours = 90 / 60;
double fee = hours * 30;
```

This produces ₹30 because integer division discards the fraction.

Correct calculation when every started hour is charged:

```
int hours = (90 + 59) / 60;
double fee = hours * 30;
```

The result is ₹60.

2.4 Reading input

Use the Scanner class for interactive input.

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);
String regNo = sc.nextLine();
int floor = sc.nextInt();
```

2.5 Alternative concept and complexity

Representation	Slot lookup	Space	Different floor sizes
Jagged array	O(F × S) worst case for scanning	O(N)	Supports naturally
One-dimensional array	O(1) for a known index	O(N)	Requires prefix offsets or a mapping when floor sizes differ
ArrayList<String[]>	O(F × S) for scanning	O(N), plus collection overhead	Supports naturally

Here, (F) is the number of floors, (S) is the maximum slots per floor and \(N\) is the total number of slots.

The jagged array is the simplest choice because each floor can contain a different number of slots.

Task 3 — Design the Control Logic

3.1 Pseudocode for slot allocation

```
ALGORITHM AllocateSlot(vehicleNo)

FOR each floor f
   FOR each slot s on floor f
      IF slot is free
         Store vehicleNo in slot
         RETURN floor f and slot s
      END IF
   END FOR
END FOR

RETURN null

END ALGORITHM
```

3.2 Pseudocode for parking fee

```
ALGORITHM CalculateFee(minutes, ratePerHour)

IF minutes < 0
   RETURN error
END IF

hours = (minutes + 59) / 60

IF hours == 0
   fee = 0
ELSE
   fee = hours * ratePerHour
END IF

RETURN fee

END ALGORITHM
```

3.3 Complete the Code

```
public class ParkingFacility {
   private final String[][] slots;

   public ParkingFacility(int[] slotsPerFloor) {
      slots = new String[slotsPerFloor.length][];

      for (int f = 0; f < slotsPerFloor.length; f++) {
         slots[f] = new String[slotsPerFloor[f]];
      }
   }

   public int[] allocate(String vehicleNo) {
      for (int f = 0; f < slots.length; f++) {
```

```
            for (int s = 0; s < slots[f].length; s++) {
                if (slots[f][s] == null) {
                    slots[f][s] = vehicleNo;
                    return new int[]{f, s};
                }
            }
        }

        return null;
    }

    public boolean release(int floor, int slot) {
        if (floor < 0 || floor >= slots.length) {
            return false;
        }

        if (slot < 0 || slot >= slots[floor].length) {
            return false;
        }

        if (slots[floor][slot] == null) {
            return false;
        }

        slots[floor][slot] = null;
        return true;
    }

    public static void main(String[] args) {
        ParkingFacility p = new ParkingFacility(new int[]{2, 1});
        System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
        System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " "
            + p.release(5, 0));
        System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```

3.4 Run the program

javac ParkingFacility.java
java ParkingFacility

Expected output:

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

3.5 Alternative concept and complexity

The current allocate() method scans slots until it finds a free one.

- Allocation: O(N) in the worst case.
- Release: O(1) when floor and slot are already known.
- Space: O(N) for the slots.

If a stack of free slot positions were maintained, allocation and release could each take O(1) time on average for the stack operations. Extra space of O(N) would be required to store the free positions, and the program would need to keep the stack consistent with slot occupancy.

Task 4 — Classes, Objects and Constructors

4.1 Fields and methods

Class	Fields	Methods
Vehicle	Registration number	getRegNo(), calculateFee()
ParkingTicket	Ticket number, vehicle, entry time	getDuration(), getTicketNo()
ParkingSlot	Floor, slot number, occupied status	occupy(), release()
ParkingFacility	Floors and slots	allocate(), release()

4.2 Constructors

A constructor creates and initialises an object.

- Default constructor: Creates an object without arguments.
- Parameterized constructor: Accepts values when creating an object.
- Overloaded constructors: Use different parameter lists for the same class.
- Constructor chaining: One constructor calls another using this().
- Copy constructor: Creates an object using another object's data.

Example:

```
class ParkingTicket {
   private final long ticketNo;
   private final String regNo;

   ParkingTicket(long ticketNo, String regNo) {
      this.ticketNo = ticketNo;
      this.regNo = regNo;
   }

   ParkingTicket(ParkingTicket other) {
      this(other.ticketNo, other.regNo);
   }
}
```

The this keyword refers to the current object. Constructor chaining avoids repeating initialisation code.

4.3 Memory

- Reference variables can be local variables or fields.
- Objects are created in heap memory.
- Local variables and method execution information are generally stored in stack frames.
- When an object is no longer reachable, it becomes eligible for garbage collection.

4.4 Alternative concept and complexity

A default constructor followed by setter calls allows an object to be temporarily incomplete. Parameterized constructors can validate required data before the object is created successfully. Static factory methods can also create and return validated objects.

Creating \(n\) simple tickets takes O(n) total time and O(n) space if all tickets are retained, regardless of which creation method is used. Parameterized constructors are the simplest option for this assignment.

Task 5 — Static Members and Object Life Cycle

Open Task05 and create Task05_Answer.

5.1 Static members

A static member belongs to the class rather than to one particular object.

```
class ParkingTicket {
   private static long nextTicketNo = 1;

   private final long ticketNo;
   private final String regNo;

   ParkingTicket(String regNo) {
      this.ticketNo = nextTicketNo++;
      this.regNo = regNo;
   }

   public long getTicketNo() {
      return ticketNo;
   }
}
```

Here, nextTicketNo is shared by all ParkingTicket objects. Every new ticket receives a new number.

Static methods are suitable for operations that do not need the data of a particular object. A static block is useful for one-time class initialisation, but it is not necessary here.

5.2 Object life cycle

1. A ticket object is created when a vehicle enters.
2. The facility stores or references the ticket while the vehicle is parked.
3. When the vehicle exits, the ticket is completed and the slot is released.
4. If no part of the program still references the ticket, the object becomes eligible for garbage collection.

The ticket number should not be reused during the intended lifetime of the ticket-number sequence.

5.3 Alternative concept and complexity

If the counter were an instance variable of ParkingFacility, two facilities could each issue ticket number 1.

Design	Counter storage for k facilities	Result
Static counter	One shared counter	One sequence shared across facilities
Instance counter	One counter per facility	Separate sequences, possibly duplicate ticket numbers

Both approaches use O(k) space for the facility objects. The counter itself uses O(1) space when static and O(k) space when stored in each facility.

If each facility must have its own numbering system, an instance counter is appropriate. If ticket numbers must be globally unique across facilities, a shared counter or a globally unique identifier is better.

Task 6 — Design the Inheritance Hierarchy

Open Task06 and create Task06_Answer.

6.1 Inheritance hierarchy

Use the following hierarchy:

Vehicle

Common vehicle details

TwoWheeler

Car

ElectricCar

Extends Car

ElectricCar extends Car, not Vehicle directly.

This is hierarchical inheritance from Vehicle and multilevel inheritance along Vehicle → Car → ElectricCar.

6.2 What is inherited?

- Vehicle stores the registration number.
- TwoWheeler defines the two-wheeler hourly rate.
- Car defines the car hourly rate.
- ElectricCar inherits the car behaviour and adds charging capability.

The super keyword calls a parent constructor or accesses a parent implementation when appropriate.

6.3 Constructor execution order

When this statement executes:

ElectricCar e = new ElectricCar("TN03C3");

The constructor order is:

1. Vehicle constructor.
2. Car constructor.
3. ElectricCar constructor.

Java initialises the parent portion of the object before the child constructor finishes.

6.4 Alternative concept: composition

Instead of subclasses, a Vehicle could contain a VehicleType object that stores its rate.

- Inheritance: Adding a vehicle type normally requires a new subclass.
- Composition: Adding a type can require only a new type or pricing object.
- Fee calculation: Both can take O(1) time for a simple rate lookup.
- Space: Both require O(1) additional information per vehicle, although composition adds a reference and may add a separate object.

Inheritance is suitable for the basic assignment. Composition becomes attractive when vehicle types and pricing rules change frequently.

Task 7 — Override and Specialise Behaviour

7.1 Method overriding

Method overriding allows a child class to provide its own version of a parent method.

For example:

```
class Vehicle {
   protected double ratePerHour() {
      return 0;
   }
}

class Car extends Vehicle {
   @Override
   protected double ratePerHour() {
      return 30;
   }
}
```

The @Override annotation helps the compiler check that the method correctly overrides an inherited method.

For overriding:

1. The method name and parameter list must match.
2. The return type must be compatible.
3. The child method cannot reduce the access level.
4. Static methods are hidden rather than overridden.

7.2 The final keyword

- A final variable cannot be reassigned.
- A final method cannot be overridden.
- A final class cannot be extended.

The vehicle registration number is a good example of a final field because it should not change after construction.

7.3 toString(), equals() and hashCode()

These methods come from Java's Object class.

- toString() provides a readable object representation.
- equals() determines whether two objects are logically equal.
- hashCode() provides a hash value consistent with equals().

If two vehicles with the same registration number are considered equal, equals() and hashCode() must both use that registration number.

7.4 Alternative concept and complexity

A single calculateFee() method could use a switch statement to check vehicle type.

- A switch with a fixed number of cases generally takes O(1) time.
- A chain of up to k type checks can take O(k) time.
- Adding a new vehicle type may require modifying the central fee method.
- Overriding keeps each vehicle's rate in its own class and generally improves maintainability.

Task 8 — Apply Polymorphism

Polymorphism means that one common reference type can work with different object types.

8.1 Compile-time polymorphism

Method overloading provides compile-time polymorphism.

double calculateFee(int minutes)

and

double calculateFee(java.time.LocalTime in, java.time.LocalTime out)

These methods have different parameter lists. The compiler selects the appropriate method based on the arguments.

8.2 Runtime polymorphism

A Vehicle[] array can contain TwoWheeler, Car and ElectricCar objects.

When v.calculateFee(130) is called, Java executes the overridden method implementation appropriate to the actual object.

Example trace:

- Declared reference type: Vehicle
- Actual object type: Car
- Executed rate method: Car.ratePerHour()
- Duration: 130 minutes
- Billable hours: 3
- Fee: ₹90

8.3 Complete the Code

```
interface Chargeable {
   double chargingFee(int kWh);
}

abstract class Vehicle {
   private final String regNo;

   protected Vehicle(String regNo) {
      this.regNo = regNo;
   }

   public String getRegNo() {
      return regNo;
   }

   protected abstract double ratePerHour();

   public double calculateFee(int minutes) {
      if (minutes < 0) {
         throw new IllegalArgumentException("Minutes cannot be negative");
      }

      int hours = (minutes + 59) / 60;
      return hours * ratePerHour();
   }

   @Override
   public boolean equals(Object o) {
      return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo);
   }

   @Override
   public int hashCode() {
      return regNo.hashCode();
   }
}

class TwoWheeler extends Vehicle {
   TwoWheeler(String regNo) {
      super(regNo);
   }

   @Override
   protected double ratePerHour() {
      return 10;
   }
}

class Car extends Vehicle {
```

```
    Car(String regNo) {
        super(regNo);
    }

    @Override
    protected double ratePerHour() {
        return 30;
    }
}

class ElectricCar extends Car implements Chargeable {
    ElectricCar(String regNo) {
        super(regNo);
    }

    @Override
    public double chargingFee(int kWh) {
        if (kWh < 0) {
            throw new IllegalArgumentException("kWh cannot be negative");
        }

        return kWh * 12;
    }
}

public class FeeDemo {
    public static void main(String[] args) {
        Vehicle[] parked = {
            new TwoWheeler("TN01A1"),
            new Car("TN02B2"),
            new ElectricCar("TN03C3")
        };

        for (Vehicle v : parked) {
            double fee = v.calculateFee(130);

            if (v instanceof Chargeable c) {
                fee += c.chargingFee(10);
            }

            System.out.println(v.getClass().getSimpleName() + " "
                + v.getRegNo() + " -> Rs " + fee);
        }

        System.out.println(new Car("TN02B2").equals(new ElectricCar("TN02B2")));
    }
}
```

Run the program
javac FeeDemo.java
java FeeDemo

Expected output:

TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true

8.4 Why does the last line print true?

Both objects have the registration number TN02B2. The equals() method checks only the registration number, not the vehicle category.

This is not necessarily correct for the parking system. Two vehicles of different categories should not normally be treated as the same vehicle merely because their registration strings match. The registration number should be unique across all active vehicles, and vehicle registration rules should be validated.

8.5 Alternative concept and complexity

If the program used instanceof checks and casting for each vehicle type, each new type could require another check.

- Dynamic dispatch: generally O(1) method selection.
- A chain of k checks: up to O(k).
- Dynamic dispatch avoids a long central chain and reduces the number of places that must change when a new subclass is added.

Task 9 — Introduce Abstraction

9.1 Abstract class

An abstract class defines common features but cannot be instantiated directly.

Vehicle should be abstract because every vehicle has a registration number and a fee calculation, but different vehicle types may have different rates.

```
abstract class Vehicle {
   private final String regNo;

   protected Vehicle(String regNo) {
      this.regNo = regNo;
   }

   public String getRegNo() {
      return regNo;
   }

   protected abstract double ratePerHour();
}
```

9.2 Interface

An interface describes a capability that a class can implement.

```
interface Chargeable {
   double chargingFee(int kWh);
}
```

ElectricCar implements Chargeable because it supports charging. Ordinary cars do not need to implement it.

9.3 Abstract class versus interface

Abstract class	Interface
Can store instance fields	Cannot have ordinary instance fields
Can have constructors	Cannot have constructors
A class extends one class	A class can implement multiple interfaces
Shares common state and behaviour	Defines capabilities or contracts

An interface can also contain default and static methods. A default method provides an implementation that implementing classes can inherit, while a static method belongs to the interface itself.

9.4 Alternative concept and complexity

If Vehicle became an interface, common instance state and constructors would need to be handled elsewhere. If Chargeable became an abstract class, a class already extending Car could not also extend it.

Both designs normally have O(1) method-call time for these simple methods. The main difference is class structure, not execution speed.

Task 10 — Packages and Encapsulation

10.1 Package structure

Organise the complete application like this:

src

parking.model

Vehicle, Car, ParkingTicket, ParkingSlot

parking.service

ParkingFacility, FeeCalculator

parking.app

Main application

Each Java file must have the appropriate package declaration at the top.

10.2 Access modifiers

Modifier	Purpose
private	Restricts access to the same class
Default	Allows access within the same package
protected	Allows access within the package and through inheritance
public	Allows access wherever the class or member is accessible

10.3 Getters, setters and validation

A getter returns a field's value. A setter updates a field after checking whether the new value is valid.

```
class ParkingSlot {
   private int slotNo;

   public int getSlotNo() {
      return slotNo;
   }

   public void setSlotNo(int slotNo) {
      if (slotNo < 0) {
         throw new IllegalArgumentException("Invalid slot number");
      }
```

```
        this.slotNo = slotNo;
    }
}
```

The slot numbering convention must be consistent throughout the program. If slots are numbered starting at 1 for users, the internal array index can still start at 0.

10.4 Wrapper classes and autoboxing

Wrapper classes represent primitive values as objects. For example, Integer wraps an int.

Autoboxing automatically converts an int to an Integer. Avoid using == to compare two Integer objects because it may compare object references rather than their numerical values. Use .equals() to compare their values.

10.5 Alternative concept and complexity

If fields were public, any calling code could modify them without validation. Every calling location would need to repeat the checks, increasing the chance of errors.

Validation of one field normally takes O(1) time. Encapsulation centralises this work, making the program safer and easier to maintain.

Task 11 — Implement, Test and Defend the Design

11.1 Testing plan

Use the following table in your report. The expected results are based on the rules in the assignment; fill the actual-result column only after running your implementation.

Test	Input or situation	Expected result
T1	Park a two-wheeler	A free slot and ticket are assigned
T2	Park a car	A free slot and ticket are assigned
T3	Park when all slots are occupied	A full-facility message is displayed
T4	Park for zero minutes	Fee is ₹0 under the started-hour rule
T5	Car parked for 60 minutes at ₹30/hour	Fee is ₹30
T6	Car parked for 61 minutes at ₹30/hour	Fee is ₹60
T7	Use an invalid slot number	Operation is rejected
T8	Enter a duplicate active registration	Operation is rejected
T9	Student with a valid monthly pass	Parking fee is ₹0
T10	Electric car uses 10 kWh	Charging fee is ₹120
T11	Release an occupied slot	Slot becomes free
T12	Release an already free slot	Operation returns false

11.2 Calculate a small day's collection

Suppose the following three vehicles park without passes:

Vehicle	Duration	Rate	Fee
Two-wheeler	90 minutes	₹10/hour	₹20
Car	130 minutes	₹30/hour	₹90
Electric car	130 minutes	₹30/hour, plus 10 kWh at ₹12/kWh	₹210
Total			₹320

11.3 Evaluate the design

The design uses:

- Encapsulation: Fields are private and validated.
- Inheritance: Vehicle subclasses share common vehicle data.
- Polymorphism: Each vehicle type provides its own rate.
- Abstraction: Vehicle and Chargeable separate common behaviour from specialised behaviour.

One limitation is that a simple array-based implementation needs to scan for vehicles or tickets. A larger system would benefit from a HashMap for fast registration-number lookup.

11.4 Alternative concept and complexity

Structure	Finding a ticket	Space
Array	O(n)	O(n)
ArrayList<ParkingTicket>	O(n)	O(n)
HashMap<String, ParkingTicket>	O(1) average lookup	O(n)

For exit processing, a HashMap keyed by registration number is generally preferable because the system can find the active ticket without scanning every parked vehicle. Its worst-case lookup can be O(n), although average lookup is O(1).

Task 12 — Re-evaluate and Adapt the Design

12A. Impact Analysis

The new requirements are:

1. E-bikes pay a flat parking fee.
2. Delivery vans pay double the car rate between 9 AM and 5 PM.
3. Visitors can purchase a one-day pass.
4. The facility has five floors with different slot counts.

The following classes or components are affected:

Component	Required change
Vehicle	Support the new vehicle categories
Car	Retain the standard car rate
New EBike	Implement flat-fee pricing
New DeliveryVan	Implement time-dependent pricing
Pass-related classes	Support one-day and monthly passes
ParkingFacility	Accept the new floor configuration
Fee calculation	Support the new pricing rules

Different slot counts require a jagged array rather than assuming every floor has the same number of slots.

12B. Evaluate the original design

The original inheritance design can be retained for vehicle identity and shared behaviour. However, placing every pricing rule directly inside vehicle subclasses would make future changes harder.

For example, the delivery van rate depends on the time of day. Adding more time-based rules to each vehicle subclass could cause duplicated code.

A better design separates the pricing rules from the vehicle classes. Existing tested classes can then remain unchanged when new pricing rules are introduced.

12C. Adapt the design

A revised structure can look like this:

Vehicle

TwoWheeler

Car

DeliveryVan

EBike

PricingRule

Separate fee calculation

ParkingPass

Monthly and one-day passes

ParkingFacility

Manages floors and slots

Example code for the new vehicle categories

```
import java.time.LocalTime;

abstract class Vehicle {
   private final String regNo;

   Vehicle(String regNo) {
      this.regNo = regNo;
   }

   public String getRegNo() {
      return regNo;
   }

   public abstract double calculateFee(int minutes, LocalTime entry, LocalTime exit);
}

class Car extends Vehicle {
   Car(String regNo) {
      super(regNo);
   }

   @Override
   public double calculateFee(int minutes, LocalTime entry, LocalTime exit) {
      int hours = (minutes + 59) / 60;
      return hours * 30;
   }
}

class EBike extends Vehicle {
   EBike(String regNo) {
      super(regNo);
   }

   @Override
   public double calculateFee(int minutes, LocalTime entry, LocalTime exit) {
      return 15;
   }
}

class DeliveryVan extends Vehicle {
   DeliveryVan(String regNo) {
      super(regNo);
   }

   @Override
   public double calculateFee(int minutes, LocalTime entry, LocalTime exit) {
      int hours = (minutes + 59) / 60;
      double fee = hours * 30;
```

```
        if (entry.isBefore(LocalTime.of(17, 0))
                && exit.isAfter(LocalTime.of(9, 0))) {
            fee *= 2;
        }

        return fee;
    }
}

public class NewVehicleDemo {
    public static void main(String[] args) {
        Vehicle bike = new EBike("TN01EB1234");
        Vehicle van = new DeliveryVan("TN02DV5678");

        System.out.println(bike.getRegNo() + " -> Rs "
            + bike.calculateFee(60, LocalTime.of(10, 0), LocalTime.of(11, 0)));

        System.out.println(van.getRegNo() + " -> Rs "
            + van.calculateFee(60, LocalTime.of(10, 0), LocalTime.of(11, 0)));
    }
}
```

Run it:

```
javac NewVehicleDemo.java
java NewVehicleDemo
```

Expected output:

```
TN01EB1234 -> Rs 15.0
TN02DV5678 -> Rs 60.0
```